



IBM BOB 2.0 HACKATHON · SEP 27, 2026

NodeFlow

The real case of a development workflow assisted by **IBM Bob 2.0**: turn an idea into a **versioned contract**, have Bob implement it, and audit the delivery with tests and an independent review.

Tauri · Rust · React Flow · TypeScript · public repo · web demo, nothing to install

An ambiguous request does not produce a verifiable deliverable — and rework gets paid three times

What it costs today

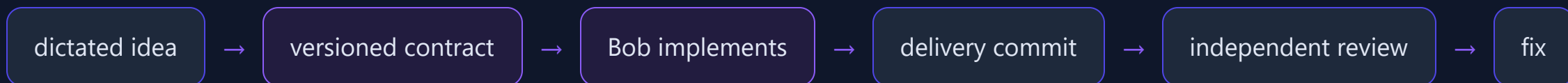
- **Implementing:** you build too much or too little, because the scope was never written down.
- **Reviewing:** there is nothing to compare against, so review turns into opinion.
- **Fixing:** the defect shows up late, when code already sits on top of it.

What we did differently

The idea is written as a **contract committed before touching code**. The agent implements against that contract and an **independent review** audits the delivery against the same text.

When the contract left something open — the cardinality of an event — the review caught it **before** it reached production.

From idea to code, with evidence at every step



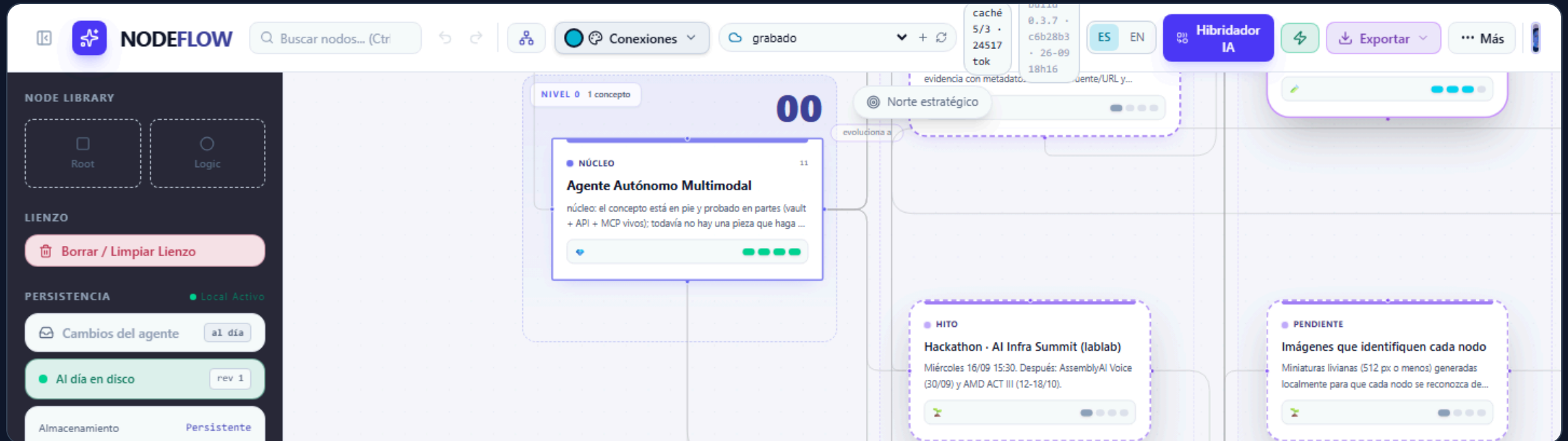
One request, complete, exactly as it sits in the git history:

- The voice segmenter contract was committed **before** any code: 00edcf6
- Bob implements the module and its endpoint: delivered in cff0baa
- The independent review finds a **real timing defect**: fixed in 0e059ed

The steps are **manual** and deliberate: this is not an automatic voice-to-code pipeline.

A canvas that draws itself while you speak

A desktop app (Tauri + Rust + React Flow): a **deterministic segmenter** on the backend decides whether what you said is an idea, a correction or noise; a **draft** appears; when the turn closes, each topic becomes a **real node** —chained, draggable and connectable— and **Ctrl+Z** reverts it. Frictionless does not mean approving first: it means being able to undo afterwards. The UI ships in **Spanish and English** (the ES/EN switch is in the top bar).



Bob wrote the voice engine. The chain proves it.

What Bob implemented

- The backend **voice segmenter** (Rust) and its endpoint.
- The **partial-result client** on the frontend.
- The **UX fixes** that came out of the first real-audio test.

Where the evidence lives

- The **contracts** committed before the code:
`docs/hackathon/PEDIDO-0*.md`
- The **task session consumption summaries**:
`bob_sessions/`
- Which files it touched and which it did not:
`docs/hackathon/EVIDENCIA-BOB.md`

What we do **not** claim: no parallelism, no subagents attributed to Bob. Only what is on the record —contracts, commits, summaries— not a line more.

Numbers you can run again

324/324

Rust tests

135/135

frontend tests

23

merged PRs

48_h

of building

One case we measured

A whole **task** handed to a coding agent produced **0 files in 80 tool calls**. The same work, split into a **piece** (one pure function, exact signature, enumerated cases), was delivered in **1 attempt** with 12 tests green.

The limits, stated up front

- 324/324 and 135/135 are **test-suite results**, not a measured saving.
- The case above is **one observed case**, not a controlled experiment.
- The process was applied **to this project**: it does not prove general productivity.

The process, reusable. The product, working.

Try it

Repo · github.com/NeonEden/nodeflow-ibm-bob

Web demo · nodeflow-ibm-bob.vercel.app (simulates voice:
without the Rust backend there is no microphone)

Desktop app · signed Windows release, or `npx tauri dev`

What we bring

A workflow where the idea is written before the code, the agent implements against that text and the delivery is audited independently — with **IBM Bob 2.0 writing the engine** and the evidence of every step in the repository.